Zowe MCP

AI-Powered z/OS Access via Model Context Protocol

Last updated: April 2026

What is MCP?

Model Context Protocol is an open standard (by Anthropic) that connects AI assistants to external tools and data sources.

⚠ The Problem

- LLMs are powerful but **isolated** from real systems
- Copy-pasting data into chat is slow and error-prone
- No structured way for AI to **act** on your behalf

✓ The Solution

- **Standardized protocol** for tool discovery and invocation
- AI reads tool schemas, decides which to call, interprets results
- Adopted by **VS Code Copilot**, **Cursor**, **Claude Desktop**, **JetBrains**, and more

Key insight: Zowe MCP lets AI assistants use z/OS tools the same way they use any other tool — and enable the use of z/OS like they can do with other systems.

What is Zowe MCP?

An **MCP server** and **VS Code extension** that gives AI assistants direct, structured access to z/OS systems and mainframe resources, such as data sets, jobs, and USS files, and do actions with them.

Built with AI: Not a single line of Zowe MCP was written manually — every line was coded with **frontier AI models in VS Code**, guided and reviewed by architects at Broadcom with experience building MCP servers and AI applications and strong mainframe background.

7	59	4	2
Components	Tools	Prompts	Resource Templates

Standalone Server

`npx @zowe/mcp-server --stdio` — works with any MCP client

VS Code Extension

Extension registers an Zowe MCP server as a **local stdio server** — used

Remote HTTP Streamable

Shared team server — HTTPS `/mcp`, optional Bearer JWT, MCP registry

Use Cases — detailed workflows

Mainframe Onboarding

New developers explore z/OS data sets, JCL, and COBOL through natural language — no ISPF knowledge needed.

AI-Assisted COBOL Development

Search source, read copybooks, compile, submit jobs, and review output — all from Copilot Chat.

Batch Job Automation

Submit JCL, wait for completion, check return codes, search spool output — driven by AI agents.

Cross-Platform Workflows

AI reads z/OS data, transforms it, writes results — bridging mainframe and distributed systems.

Code Review & Analysis

MCP prompts let AI review JCL, explain data sets, and compare PDS members with structured context.

System Administration

Run TSO and USS commands with safety guardrails, manage multiple z/OS systems from one chat.

Demo

Live Demo

Using GitHub Copilot Chat with Zowe MCP to explore data sets, submit jobs, and search COBOL source on z/OS

1. List data sets

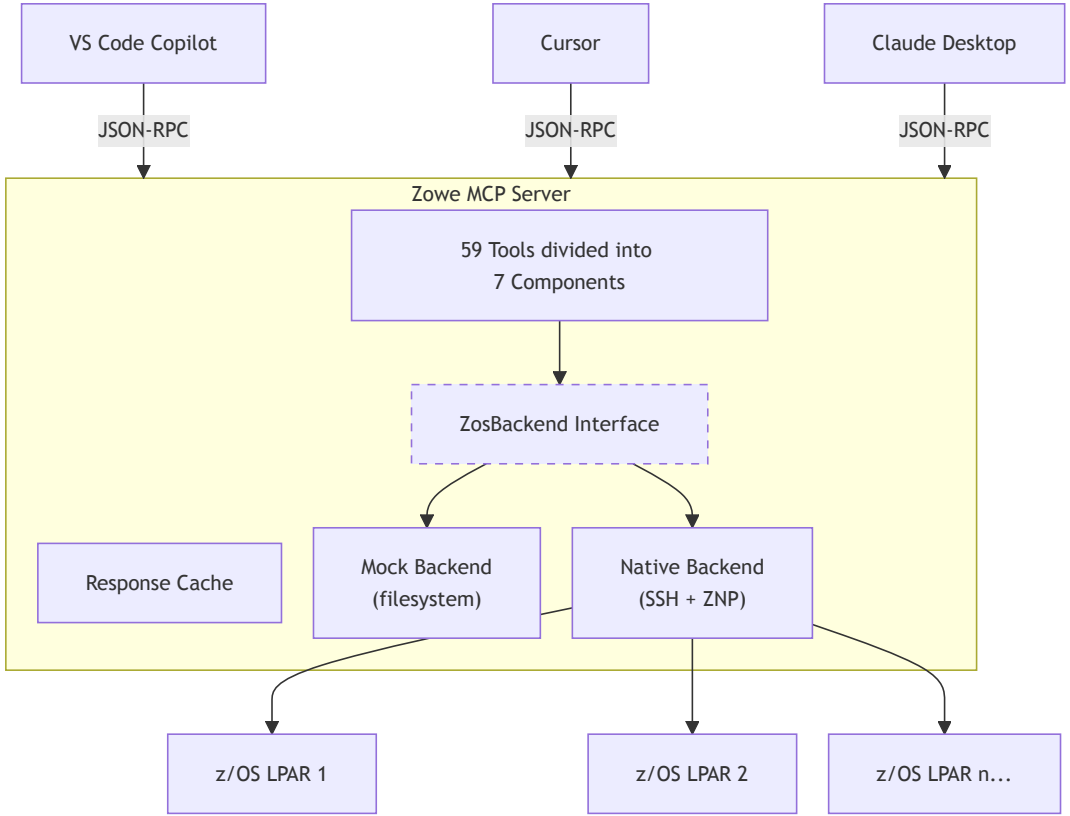
2. Search COBOL source

3. Submit & monitor job

Architecture

How Zowe MCP connects AI to z/OS

Architecture Overview



Tool Components

Zowe MCP Server — 59 tools

datasets
15
list · read · write · search
create · delete · copy · rename

uss
17
list · read · write · command
chmod · chown · chtag · copy

jobs
15
submit · status · output
cancel · hold · release · delete

context
3
listSystems · setSystem
getContext

zowe-explorer
3
open dataset · USS file · job in editor

tso
1
runSafeTsoCommand

local-files
5
download · upload · dataset · USS · job

Zowe Native Proto (ZNP)

The native backend connects to z/OS through **Zowe Native Proto** — a lightweight z/OS server deployed over SSH.

Why ZNP?

- **Only SSH required** — no z/OSMF, no APIML, no special middleware
- **Auto-deploy** — the MCP server installs and updates the ZNP binary on z/OS automatically
- **Auto-redeploy** — detects version mismatch and redeploys on the fly
- **Extensible by anyone** — open source, new operations can be added to the SDK
- **Lightweight** — small native binary, minimal z/OS footprint

How It Works

1. MCP server opens an **SSH connection** to z/OS
2. ZNP binary is **deployed to user's USS home** (if needed)
3. Commands are sent as **structured RPC** over the SSH channel
4. Results come back as **JSON** — parsed and cached by the MCP server

Key Operations via ZNP

- Data set list, read, write, search (SuperC)
- USS file operations and commands
- TSO command execution
- Job submission and management

VS Code Extension Integration

Dual Registration

- **VS Code** — `mcpServerDefinitionProviders` API
- **Cursor** — `vscode.cursor.mcp.registerServer()`
- Works in both IDEs from a single extension

Named Pipe Communication

An **additional channel** alongside stdio for deeper VS Code integration

- Bidirectional NDJSON over Unix socket
- Real-time log level changes
- Password collection via extension UI
- Zowe Explorer open-in-editor events

Extension Features

- **Mock data generation** — palette command to init mock data
- **Settings-driven** — connections, encoding, log level, job cards
  - Automatically updates the server configuration

Capabilities

What can AI do with z/OS through Zowe MCP?

Data Set Operations — 15 Tools

CRUD

- **listDatasets** — ISPF 3.4 style pattern matching
- **listMembers** — PDS or PDS/E member listing
- **readDataset** — line-windowed reads
- **writeDataset** — full or block-of-records
- **createDataset** / **createTempDataset**
- **deleteDataset** / **deleteDatasetsUnderPrefix**
- **copyDataset** / **renameDataset**
- **restoreDataset** — HSM recall

Search & Attributes

- **searchInDataset** — SuperC search, context lines, COBOL-aware
- **getDatasetAttributes** — dsorg, recfm, lrecl, SMS classes, dates

Smart Features

- **Pagination** — offset/limit with `hasMore`
- **Response cache** — LRU, 10 min TTL
- **EBCDIC encoding** — IBM-037 default, per-system overrides
- **Temp data sets** — prefix generation + cleanup

USS Operations — 17 Tools

File System

- **listUssFiles** — directory listing with metadata
- **readUssFile** / **writeUssFile** — line-windowed reads
- **createUssFile** — files and directories
- **deleteUssFile** — with recursive support
- **copyUssFile** — recursive, symlink-aware
- **chmod** / **chown** / **chtag** — permissions, ownership, encoding tags

Commands & Temp Files

- **runSafeUssCommand** — execute Unix commands with safety patterns
- **getUssHome** — resolve user home directory
- **changeUssDirectory** — per-system working directory
- **Temp operations** — getUssTempDir, getUssTempPath, createTempUssDir, createTempUssFile, deleteUssTempUnderDir

Path Resolution

- Absolute (`/u/user/...`) or relative to current working directory
- Display paths shown relative when under cwd

Job Operations — 15 Tools

Submit & Monitor

- **submitJob** — inline JCL with optional `wait: true` (adaptive polling)
- **submitJobFromDataset** — submit from PDS member
- **submitJobFromUss** — submit from USS file
- **getJobStatus** — INPUT / ACTIVE / OUTPUT + return code
- **listJobs** — filter by owner, prefix, status

Output & Lifecycle

- **listJobFiles** — spool file listing
- **readJobFile** — line-windowed spool reads
- **getJobOutput** — aggregated output across files
- **searchJobOutput** — substring search in spool
- **getJcl** — retrieve submitted JCL
- **cancelJob** / **holdJob** / **releaseJob** / **deleteJob**

Job Cards

- Auto-prepended when JCL lacks a JOB statement
- Configurable per connection (VS Code setting or config file)

TSO & Command Safety

runSafeTsoCommand

Issue TSO commands with **pattern-based safety**:

Category	Action	Examples
Dangerous	Block	DELETE SYS1.*, PASSWORD, OSHELL
Sensitive	Elicit user	DELETE own DS, SUBMIT
Safe	Allow	LISTDS, STATUS, TIME, WHO
Unknown	Elicit user	Anything not matched

runSafeUssCommand

Same safety model for Unix commands:

Category	Examples
Dangerous	`rm -rf /`, `mkfs`, `shutdown`
Sensitive	`rm`, `mv`, `chmod 777`
Safe	`ls`, `cat`, `grep`, `find`

How Elicitation Works

When a command needs approval, the server asks the MCP client to prompt the user — the AI cannot bypass this.

Multi-System Support

System Management

- **listSystems** — show all configured z/OS systems
- **setSystem** — switch active system (by host or user@host)
- **getContext** — current system, user, working directory

Connection Model

- One **system** = one z/OS host
- Multiple **connections** per host (different users)
- `system` parameter on every tool — explicit or use active

Smart Defaults

- **Single system** — auto-activated, no `setSystem` needed
- **Lazy initialization** — context created on first tool call
- **Per-system state** — each system remembers its user, USS cwd, encoding overrides

Configuration

- **VS Code** — `zoweMCP.nativeConnections` setting
- **Standalone** — `--config systems.json` or `--system user@host`
- **Mock** — `systems.json` in mock data directory

AI-Native Design

Features that make Zowe MCP work **better with LLMs** than traditional APIs:

Structured Output

- **Output schemas** (Zod) on every tool — clients validate `structuredContent`
- **Response envelope** — `_context` (resolution metadata), `_result` (pagination), `data` (payload)
- **Tool annotations** — `readOnlyHint` skips VS Code confirmation, `destructiveHint` warns

Pagination Protocol

- **List pagination** — offset/limit with `hasMore` and directive messages
- **Line windowing** — startLine/lineCount for large reads
- **Server instructions** — full protocol sent at init

MCP Prompts

- **reviewJcl** — AI reviews JCL for errors and best practices
- **explainDataset** — AI explains dataset purpose and structure
- **compareMembers** — AI diffs two PDS members
- **reflectZoweMcp** — AI reflects on usage, creates AGENTS.md

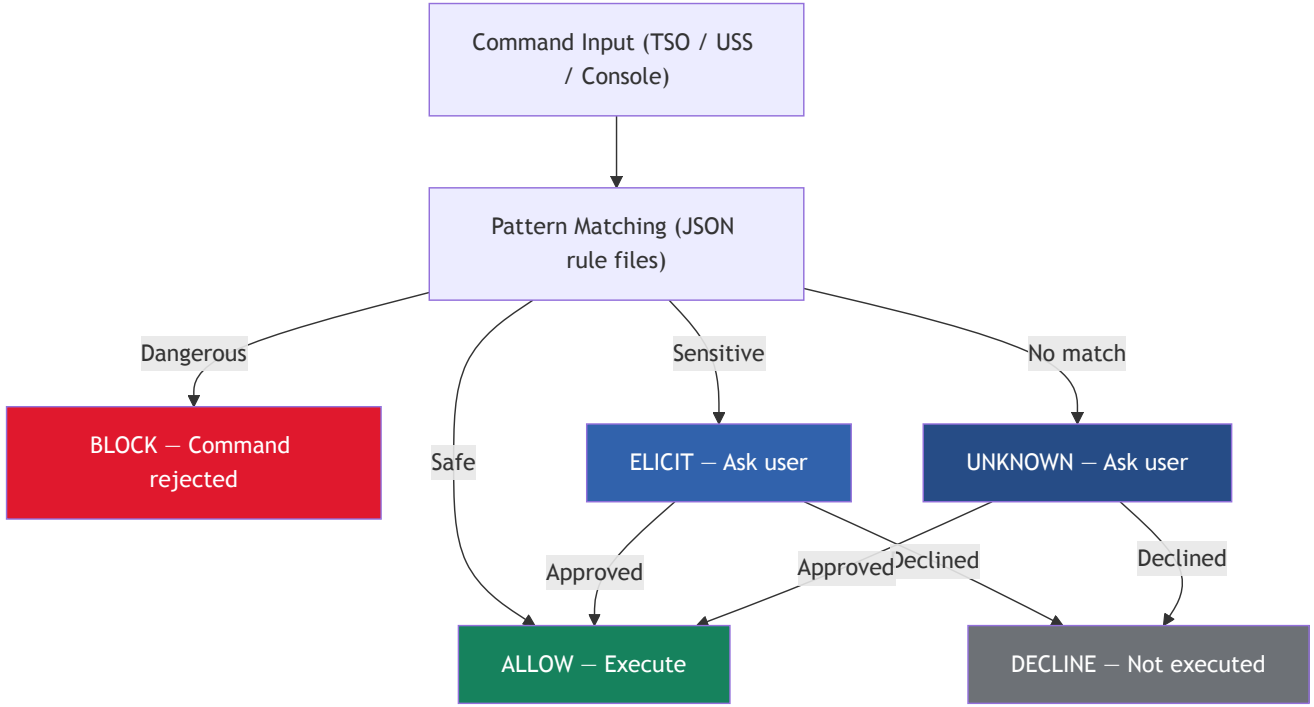
Progress Reporting

- Real-time progress via `_meta.progressToken`
- Human-readable titles: "List members of SYS1.MACLIB"
- Backend subactions: "Connecting via SSH", "Deploying ZNP"

Safety & Security

Protecting z/OS from unintended AI actions

⚠ Command Safety Model



The AI **cannot bypass** elicitation — the MCP client (VS Code / Cursor) handles the user prompt.

Credential Management

Password Collection Flow

1. Tool needs credentials for a system
2. Check if password is already known
3. If not — **request via pipe** (VS Code extension prompts user with masked input)
4. If pipe not connected — **MCP elicitation** (client prompts user)
5. On success — **store** in VS Code SecretStorage
6. On failure — **blacklist** invalid password

Concurrent Protection

When multiple tools request the same credential simultaneously, only one prompt runs — others wait.

Security Features

- Passwords **never stored in plain text** — VS Code SecretStorage (OS keychain)
- **Invalid password blacklisting** — prevents repeated failed auth
- **"Clear Stored Password"** command in VS Code palette
- Shared secret key convention: `zowe.ssh.password.${user}.${host}`

Connection Types

Mode	Credential Source
VS Code	SecretStorage + pipe prompt
Standalone	Environment variables
Mock	`systems.json` credentials

Developer Experience

Getting started, testing, and extending Zowe MCP

Getting Started

VS Code Extension

1. Install **Zowe MCP** extension
2. Set `zoweMCP.nativeConnections`
3. Open Copilot Chat — done!

Mock Mode

1. Set `zoweMCP.mockDataDirectory`
2. Run **"Generate Mock Data"** command
3. Full z/OS simulation, no mainframe

Standalone Server

```
npx @zowe/mcp-server init-mock \
  --output ./mock-data

npx @zowe/mcp-server --stdio \
  --mock ./mock-data
```

```
npx @zowe/mcp-server --stdio \
  --native --system user@host
```

Quick Tool Testing

```
npx @zowe/mcp-server call-tool \
  --mock=./mock-data \
  listDatasets \
  dsnPattern="USER.*"
```

MCP Inspector

```
npm run inspector
# Opens web UI at :6274
```

Eval-Driven Development

Every tool change is validated with **before/after AI evaluation runs**.

How It Works

1. Define **question sets** in YAML — natural language questions with assertions
2. Run evals across **multiple LLM models**
3. Compare pass rates — keep improvements, revert regressions
4. Track results in **scoreboard** (`docs/eval-scoreboard.md`)

Question Set Types

- **naming-stress** — CLI phrasing, z/OS jargon, ISPF vocabulary
- **description-quality** — pagination, search options, attributes
- **sms-allocation** — SMS parameters, JCL-style allocation
- **mutations** — write/delete flows

Example Question (YAML)

```yaml
- question: How many members does USER.INVNTORY have?
  assertions:
    - toolCall:
        tool: listMembers
        args:
          dsn: USER.INVNTORY
    - answerContains: { pattern: "2,?000" }
```

Key Findings

- **Parameter descriptions** matter more than parameter names for LLMs
- **Expanding z/OS jargon** in descriptions improved pass rates by +9.1%
- Pagination awareness remains a challenge

Extensibility — Future Directions

The MCP ecosystem is already extensible — anyone can build and register an MCP server with AI assistants, and some vendors already offer z/OS-related MCP servers independent of Zowe. Zowe MCP is focused on **core z/OS functionality** now. Here are options we're considering:

OPTION 1

Shared Library / SDK

Extract common building blocks from Zowe MCP into a **shared SDK** that other z/OS MCP servers can build on — reducing divergent approaches across the ecosystem.

Response envelopes, pagination, safety patterns, encoding, `ZosBackend` interface

OPTION 2

Zowe MCP Plug-ins

A **plug-in model** for the Zowe MCP server itself — similar to the CLI. Third parties register additional tools, prompts, and resources directly into the server.

AVAILABLE NOW ✓

CLI Plug-ins → MCP Tools

The **Zowe CLI Plugin Bridge** is implemented — any Zowe CLI plug-in can contribute MCP tools via a declarative YAML definition. First vendor plugin: **Endevor** (Broadcom).

These options are **not mutually exclusive** — they can be pursued independently or combined.

+ Extending Zowe MCP — Internals

Adding a Tool

1. Create file under `src/tools/<component>/`
2. Export `register<Component>Tools(server, deps, logger)`
3. Register in `server.ts`
4. Add output schema (Zod)
5. Add tests in `__tests__/`

Tools use `registerTool()` with:

- camelCase names
- `readOnlyHint` / `destructiveHint`
- Progress reporting
- Pagination helpers

Adding a Backend

Implement the `ZosBackend` interface:

```
interface ZosBackend {
  listDatasets( ... )
  listMembers( ... )
  readDataset( ... )
  writeDataset( ... )
  searchInDataset( ... )
  // ... 20+ methods
}
```

Current backends:

- **FilesystemMockBackend**
- **NativeBackend** (SSH + ZNP)

Adding Events

1. Define event type in `events.ts`
2. Add to union type
3. Handle in `event-handler.ts`

Event types:

- `log`, `notification`
- `request-password`
- `store-password`
- `systems-update`
- `open-*-in-editor`
- `ceedump-collected`

Zowe CLI Plugin Bridge

Expose any **Zowe CLI plug-in** as MCP tools via a declarative **YAML definition** — no TypeScript required.

YAML-Driven Tool Definitions

- Define tools, parameters, and profile types in a **single YAML file**
- **No plugin-specific TypeScript** — all details are declarative
- Each tool invokes `zowe ... --rfj` under the hood
- **Description variants** — `cli`, `intent`, `optimized` for LLM tuning

Named Profiles

- **Connection** profiles — host, port, credentials per plugin
- **Location** profiles — per-tool context (e.g. Endevor env/stage)
- Auto-generated management tools: `listConnections`, `setConnection`
- **Hot-reload** — VS Code settings immediately take effect

Built-in Smarts

- **Auto-discovery** — drop a `*.yaml` into `cli-bridge-plugins/`
- **Pagination** — list (offset/limit) and content windowing built-in
- **Response cache** — full CLI output cached, paging slices it
- **Fatal error pattern** — LLM stops on config errors, no retries

Vendor Extension Directory

```
vendor/<vendorName>/cli-bridge-plugins/*.yaml
```

Vendor-specific plugins live outside the core repo, auto-discovered at startup alongside built-in plugins.

CLI Bridge — From Zowe CLI Plugin to MCP Tools

Adapt an **existing** Zowe CLI plugin without new TypeScript: one **metadata pipeline** and one **hand-authored** tools file per plugin.

`docs/how-to-add-cli-plugin.md` (in the Zowe MCP repo) describes the whole bridge lifecycle: bridge vs native backend; auto-generated **commands YAML** and hand-authored **MCP tools YAML** (profiles, tool defs, `zowe … --rfj`, pagination, fatal vs retryable errors); then smoke-testing, LLM-tuned descriptions, evals, E2E tests, and vendor documentation — framed as a step-by-step guide for **AI coding assistants** and reviewers.

AI assistant · **You** input · **Together** draft + review

#	Step	Role
1	Install the plugin	AI
2	Generate CLI commands YAML	AI
3	Review the metadata	AI
4	Gather environment specifics	You
5	Define use cases and eval drafts	Together

#	Step	Role
6	Author MCP tools YAML	Together
7	Smoke-test	Together
8	Optimize descriptions	AI
9	Run evals	Together
10	Harden and ship	Together

Two Files: Commands YAML vs MCP Tools YAML

Aspect	CLI commands YAML (`*-commands.yaml`)	MCP tools YAML (`*-tools.yaml`)
Who creates it	Generator — `npm run generate-cli-bridge-yaml`	You (with optional AI assist)
Edit by hand?	**No** — regenerate when the plugin changes	**Yes** — defines MCP surface area
Contains	Every group, command, positional, option, alias, CLI description	`plugin`, `profiles`, `tools[]`, `zoweCommand`, pagination, error behavior
Used for	`$.endevor.list.elements.description` style **JSON references**	Runtime: bridge builds `zowe … --rfj` and wraps JSON

Bridge vs native ZosBackend

- CLI already wraps the API you need → **CLI Bridge**
- Need huge throughput or no `--rfj` → **native path** (or extend CLI first)

Profiles in tools YAML

- **Connection** — reach the service (host, port, protocol, …)
- **Location** — domain context (e.g. env/stage or Db2 `database`); often `perToolOverride: true`
- Auto tools: `listConnections` / `setConnection`, `listLocations` / `setLocation`

Wiring, Safety, and LLM-Friendly Defaults

Enable and configure

- **VS Code** — `zoweMCP.enabledCliPlugins`, `zoweMCP.cliPluginConfiguration` (profiles; hot-reload)
- **Standalone** — `--cli-plugin-enable`, `--cli-plugin-configuration name=file.json`, `--cli-plugins-dir`
- Passwords — env vars only; never commit secrets into YAML

Pagination and cache

- Opt in per tool: `pagination: list` or `content` (same envelope as core z/OS tools)
- Full CLI JSON/text cached; MCP params slice the result

Errors the LLM should not "retry away"

- Default: CLI failure → **fatal configuration** message with `stop: true`
- **Execution errors** (bad SQL, wrong element name) — set `fatalOnCliError: false` or plugin-level `retryableErrorPatterns` / `connectionErrorPatterns`

Quality loop

- Eval question sets (`packages/zowe-mcp-evals`) assert tool choice and args
- **Start small** — discovery + one read path, then add search/mutations

Same conventions as core tools: **camelCase** tool names, `readOnlyHint` where appropriate, description variants `cli` / `optimized`, and optional `displayName` for user-facing errors.

Shared HTTP server and OAuth

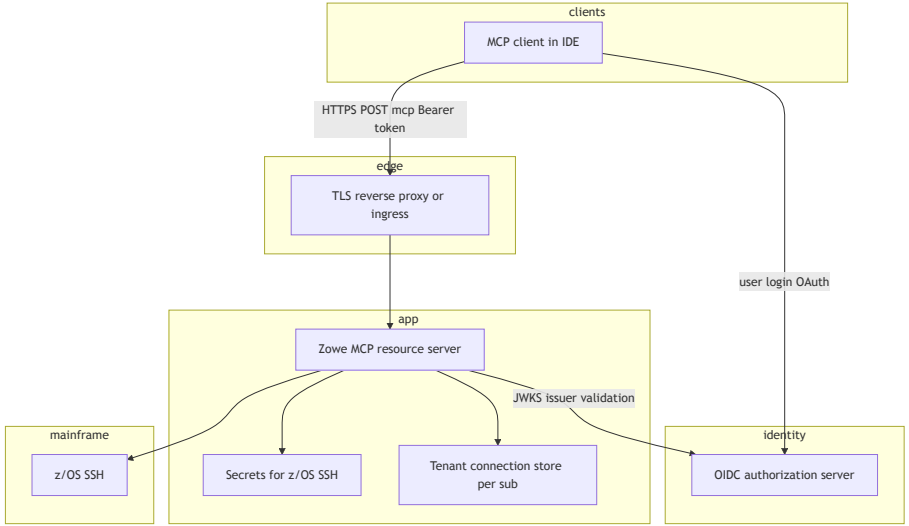
Streamable HTTP on `/mcp` — multi-session MCP over HTTPS.

OAuth and tokens

- **OAuth 2.0 resource server** — Zowe MCP does not host login pages or issue tokens; your OIDC IdP (Identity Provider) is the authorization server.
- The IdP is only for **tokens and JWKS**.
- Clients obtain access tokens (browser code flow or device flow), then send `Authorization: Bearer` on every `POST /mcp` call.
- JWT validation via `ZOWE_MCP_JWT_ISSUER` and `ZOWE_MCP_JWKS_URI`. RFC 9728 metadata helps MCP clients discover the IdP.

Identity vs z/OS

- Tenant data keyed by OIDC `sub`; not shared secrets alone.
- Mainframe SSH passwords stay separate — env, vault, or



MCP registries

An **MCP registry** is a **catalog** where **publishers** register MCP servers and **clients** (for example VS Code) **discover** them — similar to a package index, but entries describe **how to run or connect** to a server, not only source code. The registry stores **versioned metadata** so users can **browse**, **install**, or **attach** to a remote URL from the IDE gallery instead of pasting ad hoc config.

- **What you get** — Searchable listings, **trust boundaries** (official vs private org registry), and a standard `server.json` shape so tools know whether to spawn **stdio** (`npx`, Docker, …) or open **Streamable HTTP** with required **headers** (for example OAuth).
- **Where it lives** — The public MCP registry, a **vendor** catalog, or your **company's** private registry URL (large shops often host **one catalog per division** with different hostnames).
- `server.json` **per entry** — `packages`: npm tarball, PyPI, Docker, … for **local** MCP; `remotes`: **HTTPS** base URL + `type: streamable-http` and `Authorization` header description for **shared** team servers.
- **On premises** — Each Zowe MCP deployment publishes **its own** HTTPS endpoint; there is **no** single global URL — metadata documents **headers**, **OAuth**, and path (usually `/mcp`).

> **Further reading:** remote-http-mcp-registry.md (registry registration, stdio + remote together), mcp-registry-research.md (ecosystem, galleries); mcp-authentication-oauth.md (OAuth, Copilot, z/OS credentials).

Roadmap & Community

What's Next

- **z/OSMF backend** — REST API alternative to SSH
- **OAuth / MFA support** — enterprise authentication via Zowe API Mediation Layer (API ML)
- **Console commands** — z/OS operator console (code ready, waiting for ZNP support)
- **More prompts or skills* — JCL generation, COBOL analysis, batch job templates
- **Resource subscriptions** — real-time data set change notifications

Get Involved

- **GitHub** — github.com/zowe/zowe-mcp - *coming soon*
- **npm** and **VS Code Marketplace** — not available yet
- **Zowe Slack** — `#zowe-mcp` channel - *coming soon*

License

Eclipse Public License 2.0 (EPL-2.0)

Part of the **Zowe** project under the **Open Mainframe Project** (Linux Foundation)

Thank You!

Questions & Discussion

GitHub
github.com/zowe/zowe-mcp

zowe.org
zowe.org